

Airbnb Pricing and Market Intelligence Platform

INF2006 Cloud Computing and Big Data Team Project

Deployment region: AWS Asia Pacific (Singapore), ap-southeast-1

Evidence date: 23 September 2026

Submission note: team names, student IDs, contribution ownership, and personal reflections must be completed by the team before submission. They are not invented in this technical draft.

1. Problem, users and success criteria

Airbnb hosts and prospective hosts must choose a nightly price in markets that differ by location, room type, property characteristics, capacity, host characteristics, and review history. A single global average is not useful because the supplied dataset covers ten cities and prices are recorded in each city's local currency. Users also need to understand the surrounding historical market rather than treating a model output as an objectively correct price.

The primary user is a host or prospective host exploring a listing configuration. The main workflow accepts supported listing attributes, validates them in the browser and backend, runs a trained regression pipeline, and returns an estimated nightly price with its local currency and an explicit disclaimer. A signed-in user can save that estimate and later retrieve only their own history. A public market view provides city-level listing counts, median prices, and ratings from the processed analytical dataset.

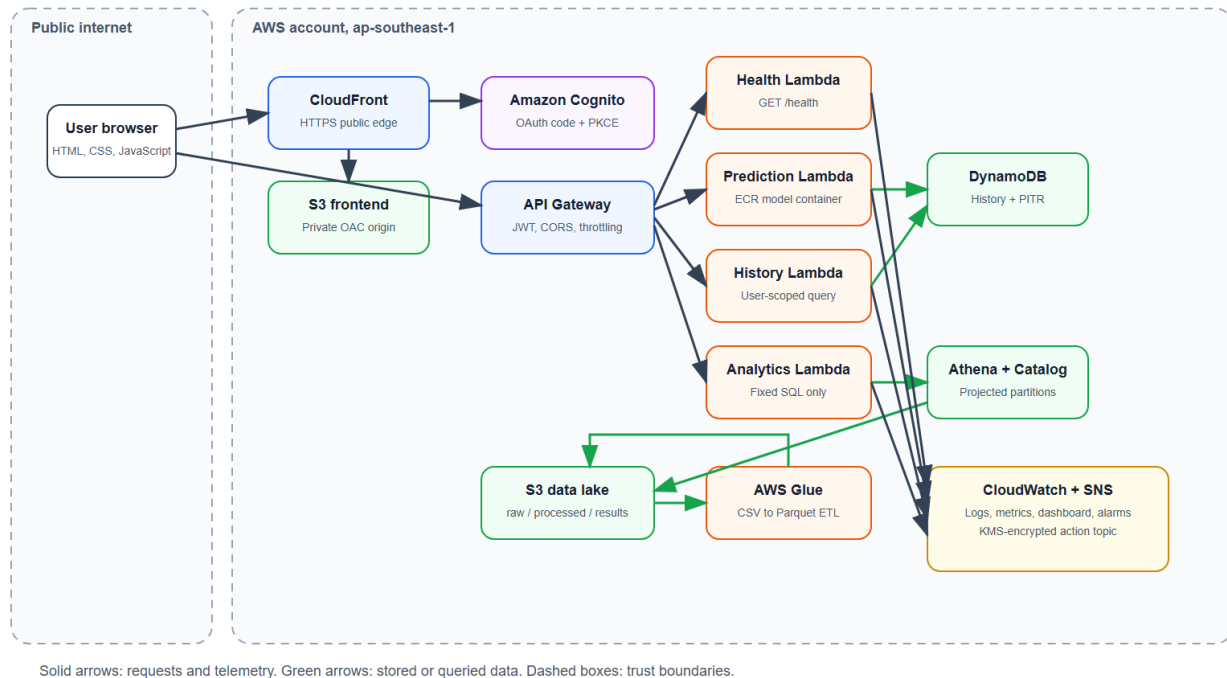
The project defines success as a working, reproducible, and evidence-backed workflow rather than maximum feature count. Functional success requires a CloudFront page, healthy API, real model inference, ten-city analytics, and authenticated history routes. Data success requires a profiled source, reproducible preprocessing and training, held-out metrics, and a cloud transform that produces queryable Parquet. Security success requires private S3 origins, HTTPS, least-privilege roles, Cognito JWT authorization, input validation, encrypted persistent stores, and evidence against named threats. Operational success requires structured logs, metrics, alarms, recovery configuration, and an honestly interpreted load test.

The service is deliberately scoped. Predictions cover the typical market up to a city-specific 99th-percentile boundary learned from training data. They are not valuations, guarantees, or financial advice. Analytics values use different local currencies and cannot be compared as though all values were denominated alike. Success does not mean eliminating model error, handling unlimited traffic, or proving production readiness from one university development account.

2. Solution overview and architecture

Airbnb Pricing and Market Intelligence Platform

Serverless AWS architecture, trust boundaries, application flow, analytics flow, and observability



The solution uses managed and serverless AWS services. CloudFront is the only public frontend delivery layer and reads static HTML, CSS, JavaScript, and configuration objects from a private S3 bucket through Origin Access Control. Browser requests reach an API Gateway HTTP API. Public routes provide health, prediction, and market analytics. Cognito provides sign-up and sign-in through the hosted UI using the authorization-code flow with PKCE. API Gateway validates JWTs for saved predictions and history.

Four focused Lambda functions separate health, prediction, analytics, and history responsibilities. The prediction function is a Lambda container image in ECR because the compatible NumPy, pandas, SciPy, scikit-learn, and model bundle exceed a practical small ZIP deployment. The history function queries DynamoDB by the verified Cognito subject. The analytics function runs one fixed Athena aggregate query, so clients cannot submit SQL.

The analytical data path uses a separate private S3 data lake. The uploaded raw listing CSV remains under `raw/listings/`. An AWS Glue 5.0 Spark job selects documented fields, rejects invalid required values, applies the same typical-market boundary used by modelling, and writes city-partitioned Parquet under `processed/listings/`. A Glue Catalog table uses partition projection for the ten observed cities. Athena runs in an enforced workgroup with encrypted output, CloudWatch metrics, a 1 GiB scan cutoff, and seven-day query-result expiry.

The architecture has two explicit trust areas: the public browser and the AWS account. HTTPS protects data in transit. IAM execution roles constrain each Lambda and the Glue job to required resources and operations. CloudWatch receives logs and metrics; three alarms target an encrypted SNS topic. DynamoDB point-in-time recovery provides a managed application-record recovery mechanism. The full labelled diagram is stored at `evidence/architecture.png`.

3. Cloud service/deployment choices and trade-offs

The deployment model is public cloud. The service model combines managed platform services and function-as-a-service. Terraform was selected so resources, policies, limits, and outputs are versioned and reviewable. This boundary reduces server maintenance but creates AWS coupling and requires careful control of provider-managed behavior and quotas.

Lambda and API Gateway were selected instead of EC2 with an Auto Scaling Group and a reverse proxy. The workload is request-driven, low-volume, and uneven, so continuously running virtual machines would add patching, capacity planning, and idle cost. Lambda provides managed horizontal execution and per-request billing. Its trade-offs are cold starts, deployment package constraints, account concurrency limits, and a maximum request duration. The measured scientific Python cold start motivated 2 GB memory and a container image. A development-account concurrency quota of 10 also limited the stress test.

S3 plus CloudFront was selected instead of a continuously running web server. The frontend is static, and CloudFront provides HTTPS delivery while Origin Access Control keeps the bucket private. The trade-off is cache invalidation after object updates. The deployment guide therefore includes an explicit invalidation command, and the smoke test verifies every asset's content type to catch cached fallback pages.

DynamoDB was selected instead of a relational database because the application has one dominant access pattern: retrieve recent predictions for an authenticated user in reverse time order. The partition key is `user_id`; the sort key combines an ISO timestamp and prediction ID. On-demand capacity avoids provisioned-capacity tuning for this small workload. The trade-off is that arbitrary relational queries are not supported and key design must follow access patterns.

S3, Glue, Parquet, the Glue Catalog, and Athena were selected instead of loading analytical records into the transactional table or operating a database cluster. Athena provides serverless SQL over S3 and Parquet reduces the scanned subset for the implemented query. Query cost still depends on bytes scanned, first-query latency is visible to users, and Glue has a minimum worker cost. A fixed query, scan cutoff, partition projection, and expiring outputs control these risks.

SageMaker, ECS, Kubernetes, RDS, Redis, a VPC, and NAT Gateway were not selected. They did not solve a measured V1 requirement and would increase operational and cost complexity. Training remains reproducible offline while inference is integrated into the cloud workflow.

4. Implementation, data design and security controls

Terraform provisions the frontend, API, Lambda functions, ECR repository, Cognito, DynamoDB, S3 data lake, Glue job and catalog, Athena workgroup, CloudWatch resources, SNS topic, and KMS key. Resource names share a project and environment prefix. S3 buckets enable Block Public Access, bucket-owner enforcement, encryption, and versioning. Frontend reads are restricted to the CloudFront distribution. Raw data is never committed to Git or managed as a Terraform object.

CloudFront adds a Content Security Policy, one-year HSTS, anti-framing, MIME-sniffing protection, and a strict referrer policy. The CSP limits scripts and styles to the application origin and browser connections to the regional API Gateway hostname pattern and exact Cognito domain. WAF is not added because no measured threat currently justifies its rule cost and operations.

The prediction API accepts only the final model schema. It rejects missing or unknown fields, unsupported categories, invalid city-neighbourhood combinations, non-finite values, out-of-range coordinates, and unreasonable numeric values. It returns safe JSON errors without traces. Predictions are clipped to the supported market boundary and always include a disclaimer. Listing inputs are not written to logs.

Cognito uses email sign-in and verification, a 12-character mixed password policy, token revocation, and a public client without a secret. The browser generates a cryptographically random OAuth state and PKCE verifier, uses SHA-256 for the challenge, validates returned state, exchanges the code directly, and holds the ID token in session storage. This reduces code interception and long-lived browser persistence risk, although browser script compromise would still expose an active token.

Authorization is enforced at API Gateway for POST /predictions and GET /history. Backend code derives the user ID only from `requestContext.authorizer.jwt.claims.sub`; no request parameter can select another user. Saved records include server-computed output, model version, timestamp, and a limited feature subset. Coordinates are not retained. The history role can only query the table, while the prediction role can only put items and write its own logs.

The analytics endpoint does not accept query text or view parameters. It submits a constant aggregate query in a named Athena workgroup and converts typed results to a stable response. The Lambda role is limited to that workgroup, catalog resources, processed objects, query-result objects, and its log group. The Glue role can read only the raw listing prefix and script, then read, write, or remove only processed listing objects.

CloudWatch alarms watch API 5xx responses and prediction or analytics Lambda errors. Alarm actions publish to an SNS topic encrypted with a rotating customer-managed KMS key. The key policy permits account administration and limits CloudWatch publishing to this account's named alarms. The topic has no committed recipient because email endpoints are personal deployment configuration; an operator must add and confirm one before treating it as a complete human notification path.

5. Analytics or AI/ML feature: data, method, evaluation and limitations

The dataset is the CC0 Airbnb Listings & Reviews release by mysarahmadbhat on Kaggle. Local files contain 279,712 listings and 5,373,143 review records across ten cities. Profiling records SHA-256 hashes, schemas, missingness, categories, price distribution, date coverage, and decoding issues. Reviews contain identifiers and dates but no text, so sentiment analysis was rejected. Prices are in each city's local currency.

The model target is nightly price. Selected inputs include city, neighbourhood, coordinates, property and room type, capacity, bedrooms, minimum nights, rating, host listing count, amenity count, instant bookability, superhost status, and identity verification. Identifier and free-text fields are excluded. Missing optional numeric fields are imputed. Categorical values are encoded inside the complete exported pipeline. The target uses $\log_{10}$ because price is strongly right-skewed.

A city-stratified 80/20 split protects representation across the ten markets. City-specific 99th-percentile price limits are learned from the training partition only, leaving 221,470 training rows and 55,363 held-out rows in V1 scope. A city median baseline, regularized linear model, and histogram gradient boosting model were compared. Selection uses the median per-city normalized MAE so one high-denomination currency does not dominate model choice.

Histogram gradient boosting performed best. On the held-out supported scope it achieved pooled MAE 191.836, RMSE 629.762, pooled local-currency R-squared 0.644, log-price R-squared 0.850, and median city-normalized MAE 0.592. The pooled currency metrics are reported for completeness but are less interpretable than per-city values. These are moderate results with material residual error, not evidence of high accuracy.

The deployed Glue transform creates the analytical representation independently of training. A measured run processed the 158,497,169-byte listing CSV in 90 seconds and produced ten Parquet objects totalling 1,223,651 bytes. This difference reflects selected columns, filtering, and compression, so it is not presented as a pure format benchmark. A city summary query scanned 566,077 bytes, used 574 ms of Athena engine time, and completed in 697 ms. The API returned all ten summaries in about 1.3 seconds during the recorded check.

Limitations include historical staleness, missing values, absent demand and booking outcomes, mixed currencies, unmeasured market changes, and exclusion of extreme luxury listings. The model may encode historical geographic and host-related patterns. Users should compare outputs with current local evidence and should not use the estimate as the sole basis for financial or housing decisions.

6. Testing, scalability/resilience and monitoring results

The repository contains 18 passing Python unit tests covering health responses, prediction validation and response shape, authenticated persistence, history isolation, analytics parsing and safe failures, data profiling, and training behavior. JavaScript files pass Node syntax checks. Terraform formatting and validation pass. The deployed smoke script verifies six frontend assets and MIME types, health, real prediction, ten-city analytics, malformed-input rejection, and unauthenticated 401 responses for both protected routes.

Security tests confirmed all four Block Public Access settings on both S3 use cases, anonymous object requests returned 403, and the data lake reported AES-256 default encryption. The Cognito authorization endpoint redirected to its hosted login page. ECR image scanning for deployed prediction image 1.0.3 completed with zero findings at scan time. Unit tests confirm history queries use only the verified JWT subject.

IAM policy simulation added a service-to-service authorization check. Required prediction writes, history queries, and Glue raw-listing reads returned allowed. Prediction reads of raw S3 data, history table scans, and Glue reads of the unused raw-review prefix returned `implicitDeny`. This verifies selected blast-radius boundaries without treating same-account services as implicitly trusted.

The bounded live test sent 20 health requests at concurrency two. Eighteen returned 200 and two returned controlled 429 responses in the recorded evidence run, with 86.58 ms median and 163.60 ms maximum latency. A later validation run returned 19 200 and one 429. The test passes only for 200 or 429 and fails for any backend error.

The higher-concurrency finding is intentionally retained. A 50-request, concurrency-25 stress run returned 26 200, 13 429, and 11 503 responses. CloudWatch showed the account reaching its concurrency quota of 10. Lowering the API token bucket improved controlled rejection but could not guarantee it because API Gateway throttling is best-effort. The system should not be described as high-scale ready in this account. A higher quota and repeated test are required before increasing expected load.

DynamoDB continuous backups and point-in-time recovery both reported ENABLED. A destructive restore was not performed because DynamoDB restores into a new table and would require a tested cutover. Operational monitoring includes structured Lambda events without listing-level inputs, Athena scan and timing metrics, Glue job state, a dashboard, and three alarms. An alarm transition initially exposed a KMS policy failure. After a scoped customer-managed key was deployed, CloudWatch history reported successful execution of the encrypted SNS action, and the test alarm was reset to OK.

7. Cost, sustainability and operational considerations

The architecture minimizes always-on compute. Lambda, API Gateway, DynamoDB on-demand capacity, Athena, Glue jobs, S3, and CloudFront charge primarily by usage or stored volume. The largest intentional batch cost is the Glue Spark job, which runs only when the source data changes and is limited to two G.1X workers, a ten-minute timeout, and no retries. Training remains offline to avoid an unnecessary managed training service.

Cost controls are implemented rather than only documented. The Athena workgroup enforces a 1 GiB scan limit. Query results expire after seven days. ECR retains only three images. CloudWatch log groups retain 14 days. The frontend uses low-cost static delivery. Data is partitioned and columnar. API throttling limits the development environment. Terraform supports teardown, although the data-lake bucket uses `force_destroy` to make intentional cleanup complete and must therefore be protected through normal change review.

Sustainability benefits come from avoiding idle servers and reducing repeated data scans. Parquet stores only analytical fields and the current query scans about 0.54 MiB. These choices reduce compute and storage activity for this workload, but no carbon measurement was performed and no universal sustainability claim is made. Glue startup overhead may outweigh benefits for very small one-off files; its value here is the reproducible cloud data-engineering path and integration with Athena.

Operational procedures include reviewing Terraform plans, using immutable ECR tags, invalidating CloudFront after frontend changes, running smoke tests, checking alarm state, and confirming the Glue run before expecting analytics. AWS credentials remain outside Git through the standard credential chain. Raw datasets, Terraform state, model binaries, environment files, and plan files are ignored.

Before a public or longer-lived deployment, the team should request an appropriate Lambda concurrency quota, add and confirm an SNS recipient, enable a remote encrypted Terraform backend with state locking, define ownership for alarms, schedule backup-restore exercises, and review current cloud spend. The university environment is intentionally small and should be destroyed when evidence collection is complete.

8. Team contribution, ethical considerations and reflection

Team-specific names, student IDs, roles, owned evidence, and personal reflections must be completed in `TEAM_CONTRIBUTIONS.md`, `README.md`, `project_manifest.yaml`, and this section before submission. Commit history provides a technical trace, but commit authorship alone is not sufficient evidence of balanced contribution. Each member should identify concrete artefacts, tests they personally verified, decisions they can explain, and a short individual reflection.

The project uses OpenAI Codex for repository inspection, implementation support, Terraform, tests, documentation, and deployment verification. AI-assisted output was not accepted as evidence by itself. Claims were checked with unit tests, live HTTP requests, Terraform plans, AWS CLI output, CloudWatch metrics, and source review. The use and verification process is declared in `AI_USE_DECLARATION.md`.

The source dataset is listed as CC0 on Kaggle, but it represents historical platform data and may contain host or reviewer identifiers in raw files. Raw records are excluded from Git and remain in a private encrypted bucket. Identifier and free-text fields are excluded from the model and processed analytics. Reviewer IDs are not uploaded because the implemented analytics do not need `Reviews.csv`. Prediction history stores the Cognito subject and a limited set of listing attributes; coordinates are intentionally omitted.

Responsible interpretation is central to the design. Prices are historical, city currencies differ, availability is not treated as proven demand, and review sentiment is not fabricated. The estimator communicates uncertainty through its disclaimer and documented metrics. Geographic and host features may reproduce historical market differences, so the output should support, not replace, current local research and human judgement.

The strongest engineering lesson is that managed services do not remove the need to test boundaries. CloudFront caching initially served fallback HTML for asset paths until invalidation. A prediction deployment briefly returned 503 during transition. Load testing exposed an account quota, and alarm testing exposed an encryption-policy gap. Each issue was diagnosed with service state and logs, corrected where practical, and retained in evidence where limitations remain.

References

1. INF2006 Cloud Computing and Big Data, Team Project 1 brief, 2026. 2. mysarahmadbhat, "Airbnb Listings & Reviews," Kaggle, CC0 1.0 Public Domain. 3. AWS service configuration and behavior are evidenced by Terraform files and dated CLI results in evidence/. 4. scikit-learn documentation and BSD-3-Clause licensed implementation, <https://scikit-learn.org/>.